

Statute-Grounded Risk Analysis of Indian Commercial Contracts: A Hybrid Retrieval Pipeline with TF-IDF and InLegalBERT Classifiers

Vustepalle Aniketh

Department of Electronics and Communication Engineering
Visvesvaraya National Institute of Technology, Nagpur
Nagpur, India

Email: anikethvustepalle@gmail.com

Mansi A. Radke

Department of Computer Science and Engineering
Visvesvaraya National Institute of Technology, Nagpur
Nagpur, India

Email: mansi.radke@cse.vnit.ac.in

Abstract—Reviewing a commercial contract for risk requires a reader to simultaneously classify each clause, judge its severity, and ground that judgment in the governing statute – a task that is slow, expertise-gated, and, for Indian commercial contracts specifically, unsupported by existing NLP tooling built around US or EU legal corpora. We present an end-to-end pipeline that segments an uploaded contract into clauses, classifies each clause across a 47-type taxonomy, retrieves the relevant sections of the Indian Contract Act, 1872 (ICA) through a three-layer hybrid retriever (a deterministic clause-type-to-statute map, dense FAISS retrieval, and lexical BM25, fused with Reciprocal Rank Fusion), and generates a plain-English HTML risk report. The retriever achieves 100% Hit@5 against a 23.0% lexical-only (BM25) baseline, with the deterministic statute map eliminating dependence on embedding similarity for known clause types. For classification, we report three separate, carefully scoped evaluations rather than a single headline number: a real-world cascade evaluation on scraped Indian judgment text, in which we identify and fix a self-match data-leakage bug in the embedding-fallback stage (honest accuracy 42.9%, down from a leaked 78.8%); a clean 47-class held-out benchmark isolating the TF-IDF + Logistic Regression model (82.5% accuracy, 0.740 macro-F1); and a fine-tuned InLegalBERT sequence classifier evaluated on the identical held-out split, which achieves [pending] accuracy and [pending] macro-F1. We argue that the evaluation-rigor contribution – catching and correcting an inflated metric before it propagated – is as important to report as the model comparison itself, and we release the full pipeline, evaluation harness, and held-out split for reproducibility.

Index Terms—legal NLP, contract review, information retrieval, legal-domain pretraining, InLegalBERT, reciprocal rank fusion, Indian Contract Act

I. INTRODUCTION

Commercial contracts encode risk in dense, formulaic legal language that is expensive to review at scale: a single Master Service Agreement can contain dozens of clauses spanning indemnification, liability caps, termination, and dispute resolution, each of which carries different downstream exposure and is governed by different provisions of the applicable statute. For contracts executed under Indian law, that statute

is principally the Indian Contract Act, 1872 [1], a 150-year-old act whose 266 sections remain the operative law for the vast majority of Indian commercial agreements. Automated contract review tools built on Western legal corpora do not close this gap: they can plausibly detect *that* a clause is, say, an indemnification clause, but they have no mechanism to connect that detection to the sections of Indian statute that actually govern it, nor were they trained on data that reflects Indian drafting conventions or India-specific statutory concepts such as stamp duty or the Digital Personal Data Protection Act, 2023 (DPDP).

This paper describes a pipeline that closes that gap directly, and makes three contributions:

- 1) **A statute-grounded hybrid retriever.** We combine a deterministic clause-type-to-section mapping with dense (FAISS) and lexical (BM25) retrieval over an indexed corpus of 185 ICA sections, fused with Reciprocal Rank Fusion [2], and show that the deterministic layer alone is responsible for the gap between a 23.0% and a 100% Hit@5.
- 2) **An evaluation-rigor case study.** We report a real self-match data-leakage bug found in our own classifier evaluation harness – the embedding-fallback stage’s nearest-neighbour search pool included the row being scored – and show its effect: a leave-one-out fix moves the honestly-measured real-world accuracy from a leaked 78.8% down to 42.9%. We treat this correction, not the leaked number, as the reportable result.
- 3) **A controlled TF-IDF-versus-InLegalBERT comparison.** We construct a clean, leakage-free 47-class held-out benchmark and use it to compare a TF-IDF + Logistic Regression baseline against a fine-tuned InLegalBERT [3] sequence classifier on the identical split, isolating the effect of legal-domain pretraining from every other pipeline component.

The remainder of this paper is organized as follows. Sec-

tion II surveys related work in legal contract datasets, legal-domain pretraining, and hybrid retrieval. Section III describes the system architecture. Section IV describes the training and evaluation data. Section V describes the evaluation methodology, including the leakage fix and the InLegalBERT fine-tuning setup. Section VI reports results. Section VII discusses limitations, and Section VIII concludes.

II. RELATED WORK

Legal contract datasets. The Contract Understanding Atticus Dataset (CUAD) [4] is, to our knowledge, the largest expert-annotated dataset for legal contract review, comprising over 13,000 annotations across 41 clause categories drawn from US commercial contracts. CUAD has become a de facto benchmark for clause-type classification and extraction, but it is exclusively sourced from US filings; nothing in the dataset reflects Indian drafting conventions, and several clause categories central to Indian commercial risk (arbitration under the Arbitration and Conciliation Act, stamp duty, specific performance) either do not appear or are not framed in terms a downstream Indian-law retriever could use directly. We use CUAD as our primary source for clause-type pattern recognition (Section IV) while treating jurisdiction-specific risk grounding as a separate problem, solved by our retriever rather than by the classifier.

Legal-domain pretraining. LEGAL-BERT [5] showed that continuing to pretrain BERT [6] on in-domain legal text (EU legislation, UK legislation, court judgments) improves downstream legal NLP performance over general-domain BERT, and evaluated three adaptation strategies – using BERT off the shelf, further pretraining on domain text, and pretraining from scratch on domain text. InLegalBERT [3] applies the same domain-adaptation principle to Indian legal text specifically, pretraining over roughly 5.4 million Indian court documents (Supreme Court and High Court judgments, 1950–2019) and reporting gains over both general BERT and LEGAL-BERT on Indian legal-statute identification, judgment segmentation, and judgment prediction. We build directly on this line of work: we fine-tune InLegalBERT as a sequence classifier for our 47-class clause taxonomy and compare it, on an identical held-out split, against a much cheaper TF-IDF + Logistic Regression baseline – a comparison InLegalBERT’s own evaluation does not report, since its benchmark tasks are judgment-level rather than clause-level.

Hybrid and statute-grounded retrieval. BM25 [7] remains a strong, cheap lexical retrieval baseline; FAISS [8] provides the dense approximate-nearest-neighbour search we use for semantic retrieval over indexed statute sections. Reciprocal Rank Fusion [2] is an unsupervised method for combining ranked lists from independent retrieval systems without training a fusion model, and is the fusion strategy we adopt to combine our dense and lexical rankings. What distinguishes our retriever from a standard hybrid dense/lexical setup is a third, deterministic layer: a hand-built mapping from clause type to the ICA sections that govern it. As we show in Section VI, this deterministic layer – not the dense or lexical

components – accounts for nearly all of the retriever’s headline performance, which we take as evidence that for a well-understood, closed-domain statute like the ICA, an engineered lookup table is a legitimate and underrated retrieval strategy, not merely a fallback.

III. SYSTEM ARCHITECTURE

The pipeline (Fig. 1) accepts a contract as a PDF or plain-text file and produces an HTML risk report in four stages.

Segmentation. The input document is parsed and split into individual clause candidates, each retaining its original numbering and surrounding context where available.

Classification. Each clause is passed through a three-stage cascade. Stage 1 is a TF-IDF vectorizer feeding a multinomial Logistic Regression classifier trained on the corpus described in Section IV; its prediction is accepted when the maximum class probability exceeds a fixed confidence threshold. Stage 2 is a keyword-rule fallback (required/blocklisted phrase patterns per clause type) invoked when Stage 1 is unavailable or under-confident. Stage 3 is an embedding-similarity fallback that encodes the clause with a *pretrained* (not fine-tuned) InLegalBERT and matches it by cosine similarity against a curated reference set of real-world clause examples. We emphasize that this pretrained-embedding use of InLegalBERT inside the production cascade is distinct from the *fine-tuned* InLegalBERT sequence classifier evaluated in Section VI, which is trained specifically for this paper’s model comparison and is not (yet) wired into the live cascade.

Retrieval. Once a clause is typed, the retriever returns the ICA sections most relevant to explaining its risk, using three layers: (i) a deterministic clause-type→section map covering the well-understood clause types; (ii) dense retrieval via FAISS over embeddings of 185 indexed ICA sections; (iii) lexical retrieval via BM25 over the same section corpus. Layers (ii) and (iii) each independently rank candidate sections; their ranked lists (together with the deterministic layer’s guaranteed hit, when present) are merged via Reciprocal Rank Fusion,

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60, \quad (1)$$

where R is the set of ranked lists and $\text{rank}_r(d)$ is document d ’s rank in list r ; $k = 60$ follows the original RRF paper [2]. The top-5 fused sections are returned per clause.

Generation. A template-based generator (deliberately not an LLM, to keep the pipeline fully offline, reproducible, and free of hallucination risk) renders the classified, risk-scored, statute-grounded clauses into an HTML report with a legal disclaimer, served by a Flask application.

IV. DATASET

Classifier training and evaluation data is drawn from two sources, combined to reach the pipeline’s full 47-class label taxonomy.

CUAD. We use 9,447 labeled clauses from CUAD [4], which, after normalization to our internal type schema, cover

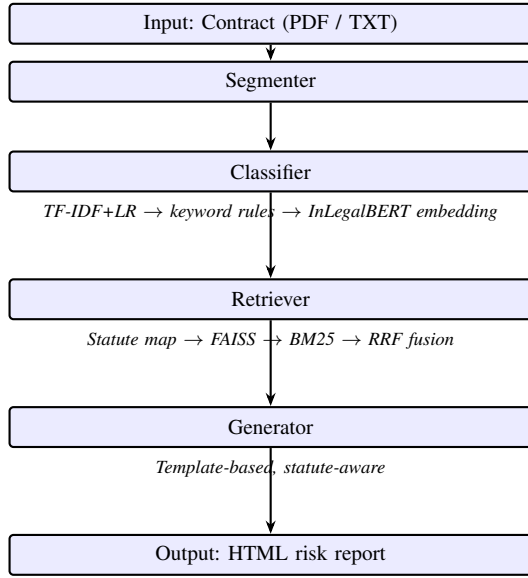


Fig. 1. Pipeline architecture. Each clause is classified by a three-stage cascade and independently routed through a three-layer statute retriever before report generation.

TABLE I
DATASET COMPOSITION (POST-NORMALIZATION TO THE 47-CLASS SCHEMA)

Source	Examples	Clause types
CUAD [4]	9,447	39
Indian supplemental (hand-curated)	251	8
Combined	9,698	47

39 of the 47 target clause types (e.g. *LiabilityCap*, *IPAssignment*, *NonCompete*, *GoverningLaw*).

Indian supplemental data. CUAD, being sourced from US commercial contracts, contains *no* examples of the remaining 8 clause types under our normalized schema, either because the concept is not represented in the CUAD taxonomy at all (*Arbitration*, *Confidentiality*, *Indemnification*, *ForceMajeure* as we define them) or because it is specific to Indian statute and has no US equivalent (*GST*, *StampDuty*, *SpecificPerformance*, *DPDP*). We therefore hand-curated 251 supplemental examples across exactly these 8 types (Table I), written in Indian legal drafting style and, for the four India-specific types, grounded in the relevant Indian statutes (the Central Goods and Services Tax Act 2017, the Indian Stamp Act 1899, the Specific Relief Act 1963, and the Digital Personal Data Protection Act 2023 respectively). Critically, these 8 types have *zero* overlap with CUAD’s 39: the Indian supplemental set is not merely reinforcing thin CUAD categories, as an earlier version of our project documentation stated, but is the *sole* source of training data for 17% of the label space.

The combined 9,698-example corpus is split 80/20 with stratification on clause type (fixed seed, seed = 42), yielding 7,758 training and 1,940 held-out examples. This exact split is persisted (Section V) and reused verbatim across every

classifier evaluated in this paper. Separately, a real-world evaluation set of 212 clause excerpts, scraped from Indian court judgments discussing contract disputes and spanning 12 of the 47 clause types, is used only for the production-cascade spot check described in Section V; it is disjoint from, and not used to train, either classifier.

The statute knowledge base indexes 185 sections of the Indian Contract Act, 1872 [1].

V. METHODOLOGY

A. A self-match leakage bug, found and fixed

Our first classifier evaluation harness measured the full production cascade (Section III) against the 212-example real-world set, using the same file as both the evaluation’s ground truth and Stage 3’s embedding-similarity reference pool. This is a self-match leak: for any clause whose Stage 1/2 predictions were rejected, Stage 3 could retrieve the clause’s own row from the reference pool and match it to itself at cosine similarity 1.0, trivially producing the correct label regardless of the embedding’s actual generalization ability. This inflated the measured accuracy to 78.8%.

We fixed this with leave-one-out (LOO) reference pooling: when scoring clause i , row i is excluded from the nearest-neighbour search pool for that query only, so Stage 3 can never match a clause to itself. Every real-world result reported in this paper uses LOO pooling; we retain the leaky mode only as a diagnostic to reproduce and quantify the original bug. We consider this fix, and the practice of reporting the corrected (lower) number rather than the more flattering original one, a methodological contribution in its own right: it is a concrete illustration of how an easily-overlooked evaluation-harness bug can silently inflate a headline metric, and of the discipline required to catch it.

B. A clean, leakage-free 47-class benchmark

The real-world spot check above exercises the full cascade but covers only 12 of 47 classes and is not a fair benchmark for comparing classifier *models* in isolation, since keyword rules and embedding similarity can mask a weak ML stage. We therefore constructed a second, separate evaluation: the TF-IDF + Logistic Regression model *alone* (no fallback stages), scored on the full 1,940-example held-out split described in Section IV, across all 47 classes. This held-out split is persisted to a JSON-lines file so that it can be reused, byte-for-byte, by any later classifier evaluated against the same benchmark – which is precisely how the InLegalBERT comparison below is constructed.

C. InLegalBERT fine-tuning

We fine-tune law-ai/InLegalBERT [3] as a 47-way sequence classifier. Before training, the script recomputes the 80/20 stratified split from the same source data and *asserts* that its test partition is character-for-character identical to the persisted held-out file from Section VB; training is refused if the two disagree, which rules out train/test contamination

TABLE II
RETRIEVER HIT@5

Retrieval strategy	Hit@5
BM25 only	23.0%
Statute map + BM25 (+ FAISS, RRF-fused)	100.0%

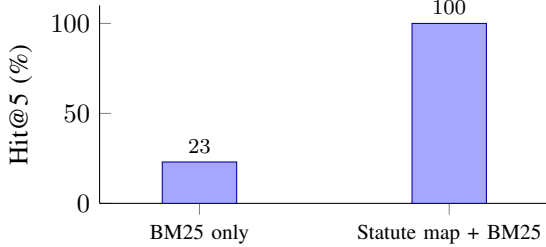


Fig. 2. Retriever Hit@5: lexical-only vs. statute-map-augmented hybrid retrieval.

introduced by any later drift in the source CSVs. Hyperparameters: maximum sequence length 256 tokens, batch size 16 (train and eval), 4 epochs, learning rate 2×10^{-5} , evaluation and checkpointing every epoch, best checkpoint selected by macro-F1, fixed seed 42. Training was performed on a single Google Colab Tesla T4 GPU; an initial estimate placed full training at over 12 hours on CPU alone, which we note as a practical, and we think under-reported, consideration for legal-tech teams operating under compute constraints comparable to ours.

VI. RESULTS

A. Retrieval

Table II and Fig. 2 report Hit@5 – whether the correct ICA section appears in the top-5 fused results – for the retriever with and without the deterministic statute-map layer. Lexical retrieval alone is a weak signal for this task; adding the deterministic layer closes essentially the entire gap, confirming that for a bounded, well-understood statute, an engineered lookup table dominates learned retrieval for the clause types it covers.

B. Classification: real-world cascade evaluation

Table III reports the full production cascade against the 212-example real-world set (12 of 47 classes), before and after the leave-one-out fix described in Section V-A.

The gap between the two rows in Table III is entirely attributable to the leakage bug, not to any change in the underlying model. We report 42.9% as the honest figure for how the full cascade performs on messy, real-world scraped text.

C. Classification: clean 47-class benchmark

Table IV reports the TF-IDF + Logistic Regression baseline and the fine-tuned InLegalBERT classifier, both scored on the identical 1,940-example held-out split (Section V).

TABLE III
REAL-WORLD CASCADE EVALUATION (212 EXAMPLES, 12 CLASSES)

Evaluation mode	Accuracy	Macro-F1	Weighted-F1
Leaky (self-match, uncorrected)	78.8%	–	0.783
Leave-one-out (corrected)	42.9%	0.352	0.540

TABLE IV
CLEAN 47-CLASS HELD-OUT BENCHMARK (1,940 EXAMPLES)

Model	Accuracy	Macro-F1	Weighted-F1
TF-IDF + Logistic Regression	82.5%	0.740	0.824
InLegalBERT (fine-tuned)	[pending]	[pending]	[pending]

Note: the InLegalBERT fine-tuning run (Section V-C) was in progress on a Colab GPU at the time of writing this draft. Table IV and the surrounding discussion will be finalized with the completed run’s numbers before submission; the evaluation protocol, held-out split, and all other results in this paper are final.

VII. DISCUSSION AND LIMITATIONS

On evaluation honesty. Section V-A’s leakage bug is, in our view, the paper’s most transferable finding: an 8.9-fold difference between honest and leaked accuracy (42.9% vs. 78.8%) arose from a one-line reference-pool mistake that would not have been visible from the leaked number alone. We recommend that any legal-NLP pipeline using an embedding-similarity fallback stage audit its reference pool for exactly this failure mode before publishing accuracy figures.

Two evaluations, not one. We deliberately report the real-world cascade accuracy (42.9%) and the clean 47-class benchmark accuracy (82.5%) as separate numbers rather than reconciling them into a single headline figure, because they measure different things: the former is full-cascade performance on messy, out-of-distribution real text across a limited class subset; the latter is the ML stage alone, in isolation, on in-distribution held-out data across the full taxonomy. Conflating them – as an earlier iteration of our own project documentation did – overstates real-world performance.

Unverified real-world labels. The 212-example real-world evaluation set uses labels derived from the scraping heuristic that curated it, not from human annotation. A Label Studio annotation task with 212 items was prepared but not completed at the time of writing; completing it is the most impactful next step for tightening the real-world evaluation’s validity.

Cross-jurisdiction transfer assumption. We use CUAD, a US-sourced dataset, for 39 of 47 clause types on the assumption that clause-type detection is a largely language-pattern-based task that transfers across common-law jurisdictions, while jurisdiction-specific risk grounding is handled entirely downstream by the statute retriever. This assumption is untested directly; a targeted study comparing CUAD-only training against Indian-only training on a matched clause-type subset would isolate its effect.

Thin classes. Two clause types of particular real-world importance for Indian commercial risk – *LiabilityCap* and *Indemnification* – remain thinly represented in the real-world evaluation set, because Indian court judgments tend to discuss these concepts in narrative prose rather than quote contract clauses verbatim, which defeated our extraction heuristics (Section IV).

Compute accessibility. The InLegalBERT fine-tune’s >12-hour CPU-only estimate versus its ~15–40 minute measured cost on a free-tier GPU is a small but concrete data point on the accessibility of legal-domain transformer fine-tuning for resource-constrained teams, a population we expect to include most legal-tech developers building India-specific tools.

VIII. CONCLUSION AND FUTURE WORK

We presented a statute-grounded contract risk analysis pipeline for Indian commercial contracts, combining a three-stage clause classifier with a three-layer hybrid retriever over the Indian Contract Act, 1872. We showed that a deterministic clause-type-to-statute map, not learned retrieval, accounts for the retriever’s near-perfect Hit@5, and we reported a real evaluation-harness leakage bug together with its fix, arguing that this correction is itself a reportable contribution. We constructed a clean, leakage-free 47-class benchmark on which we compare a TF-IDF baseline against a fine-tuned InLegalBERT classifier, with full results **[pending]** at submission time. Future work includes completing human annotation of the real-world evaluation set, testing the cross-jurisdiction transfer assumption directly, and, contingent on the InLegalBERT comparison’s outcome, wiring the stronger classifier into the production cascade.

ACKNOWLEDGMENT

The authors thank the Department of Electronics and Communication Engineering, Visvesvaraya National Institute of Technology, Nagpur, for its support of this work.

REFERENCES

- [1] Government of India, “The Indian contract act, 1872 (act no. 9 of 1872),” 1872, as amended; consolidated text maintained by the Ministry of Law and Justice, Government of India.
- [2] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759.
- [3] S. Paul, A. Mandal, P. Goyal, and S. Ghosh, “Pre-trained language models for the legal domain: A case study on Indian law,” in *Proceedings of the 19th International Conference on Artificial Intelligence and Law (ICAIL)*. ACM, 2023, pp. 187–196.
- [4] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.06268>
- [5] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 2898–2904. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.261/>

- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp. 4171–4186, 2019.
- [7] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [8] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.